Introduction: What topic have you chosen and why?
We have chosen to make a 3D First Person Shooter (FPS) game. A game was chosen because games are fun and approachable to anyone. The 3D aspect of the game will be implemented using raycasting. Raycasting works by tracing rays from a fixed point (e.g. the player) until it meets an object blocking it from going further (e.g. the map or the enemies) and a corresponding line is drawn. This is done around a certain angle around the fixed point to draw a full image. We chose to use raycasting as our method because it was feasible to implement and we can use the OpenGL library in doing so.

Related work: Where do your ideas come from, and where will you obtain additional resources (references, libraries, etc.)?
Ideas come from pre-existing games such as Doom, Wolfstein 3D.

Additional resources obtained from YouTube, we are already using the OpenGL library. We will use GitHub for version control and working as a large team.

Team: Who are the team members, and how will all the work be distributed among them?
We have 6 team members, Zhongyao, Tianchang, Yuanhao, Jiayue, Linzheng, and Praemai. Our project is divided into 3 main parts: enemy design, player design, and map design.

Zhongyuan and Tianyang will work on **enemy design**.
Yuanhao and Jiayue will work on **player design**.
Linzheng and Praemai will work on **map design**.

Our work: What will you implement in detail, e.g., the overall architecture and individual components in your program, and what are the highlights that make your project distinctive?
Game Concept
- Player role: Zoo visitor
- Game objective: Collect fruits, feed animals, find the key, and exit through the correct exit

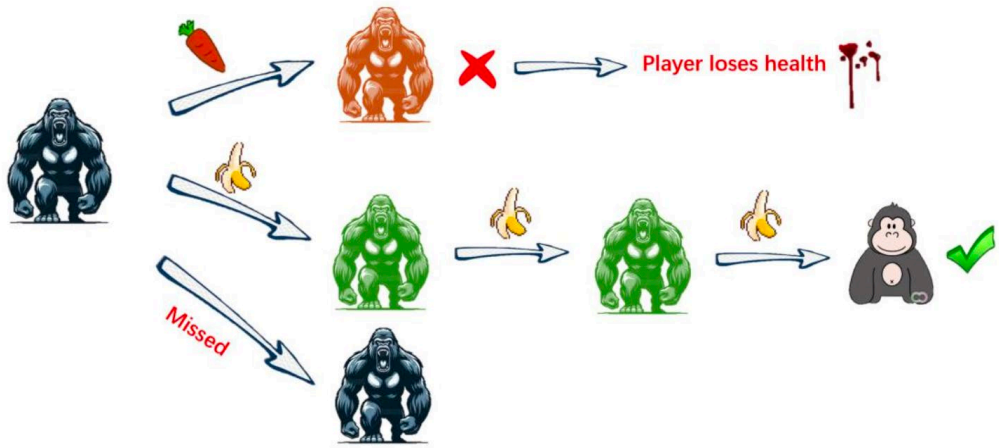
Levels & Difficulty
- Difficulty can be adjusted by animal types, numbers, and speed
- Each level contains different numbers of animals and fruits

Core Gameplay

1. Collect Fruits

- Player collects fruits scattered across the map

2. Feed Animals



- Each animal prefers a specific fruit, for example:
  - Monkey → Banana
  - Tiger → Meat
  - Rabbit → Carrot
  - Panda → Bamboo
- Correct feed: if fed correctly three times (exact number to be determined), the animal changes from fierce to cute and disappears
- Wrong feed: animal turns red (angry) and player loses health
- When a fruit is thrown, a straight line is calculated and we check whether a correct animal is hit or not
- If a correct animal is hit, then all animals around the straight line will stop and wait for the fruit (they will not get the fruit) until the animation finishes playing and the correct animal is fed

3. Animal Movement & Animation

- Each animal has 2–3 front-facing running sprites (fierce version)
- Sprites loop when the animal approaches the player to simulate running
- One red sprite for wrong feed
- One green sprite for wrong feed
- One cute sprite for consecutive three correct feeds

Items & Rewards

- Fruits: one sprite per type
- Key: one sprite to complete the level

Game end condition

- Player finds the key and exits through the correct door

Future enhancements

- Add walking sound effects for different animals

Overall architecture and individual components:

Mob (parent class):

- Health
- Speed
- Location
- Facing in which direction

Player (extends mob):

- Field of view (how many degrees of vision)
- Weapon
- X-ray vision (skill tree upgrade)

Enemy (extends mob):

- Reward given (restore player health, increase player damage, etc)
- Weapon
- Track player (find shortest path to player)

Weapon (parent class):

- Damage
- Range
- Cooldown time
- Ammo (how many uses before weapon runs out)
- Hit detection method

UI:

Game states

- Start game

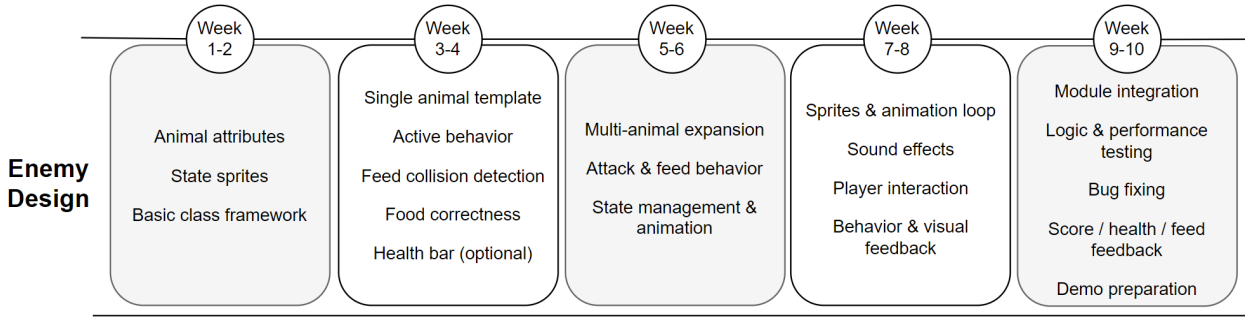
- Load game save file
- Pause game
- Exit game
- Enter developer view (shows all data on top of the game, e.g. the array indices the player is on, enemies, etc)

Game view
- Player
  - Health bar
  - Weapon damage
  - Weapon cooldown bar
  - Weapon ammo
- Enemy
  - Health bar

Save file data:
- Player class
- Enemy class
  - Type
  - Amount
  - Health
- Map

Schedule: What are the project milestones (at least bi-weekly) and their respective completion dates?

Roadmap



Week 0-2

Map Design:

- Create GitHub repository

- Design the map
- Add basic features
  - Player movement
  - Player vision
  - Collision between player and wall
  - Wall textures

Enemy Design：

Basic Framework

- Determine animal types, attack methods, calming items, health, movement speed, etc.
- Draw pixel sprites for different animal states (fierce version + wrong feed red/green version + cute version)
- Build the basic Animal class, adding core attributes and state variables

Player Design：

- Determine attack pattern
- Build hit detection method
- Solve the problem of the trajectory of the bullets (fruits)

Week 2-4

Map Design：

- Add health regeneration points to map
- Add ammo regeneration to map

Enemy Design：

Single Animal Behavior + Player Interaction

- Choose one animal as a template and implement active behavior: detect player → move → attack → get fed → disappear
- Player food throw and animal collision detection
- Check correctness of food type
- Optional: simple health bar display

Player Design：

- Decide the type of weapons
- Create animations for weapons and bullets
- Create the weapon picking-up mechanism

Week 4-6

Map Design：

- Implement saving game state
- Implement loading save file into game

Enemy Design：

Multi-Animal Expansion

- Create different animal types based on the template
- Improve different animals' attack methods and feeding responses
- Maintain unified state management and animation switching

Player Design：

- Decide other kinds of supplements
- Create player hit feedback
- Add sound effects (movement, hitting feedback, picking-up, shooting, gaining or losing health, etc.)

Week 6-8

Map Design：

- UI design

Enemy Design：

Sprites, Sound Effects, and Interaction Enhancement

- Add animal sprites and simple animation loops
- Add sound effects (movement, feeding feedback, etc.)
- Improve interaction with player throwing/weapon logic
- Optimize animal behavior and visual feedback

Player Design：

- Balancing game playability
- Adjust the attack values of weapons

- Adjust the damage received by players
- Adjust the maximum number of weapons that can be held
- Adjust the character's speed, and the flight speed of weapons
- Communicate with the enemy design team to determine the final game values and balance

Week 8-10

Map Design：

- Playtesting
- Bug fixing

Enemy Design：

Integration, Testing, and Debugging

- Integrate all modules (player, animals, food, UI)
- Test logic and performance when multiple animals appear simultaneously
- Fix bugs, improve score/health/feeding feedback
- Prepare demo version

Player Design：

- Playtesting
- Bug fixing